



Clinic Appointment and Prescription Manager

Programming for Problem Solving (PSP) – Java

Presented by: Sundi Manilatha

Abstract

The **Clinic Appointment and Prescription Manager** is a console-based Java application that manages patient information, appointments, and prescriptions using fundamental programming concepts.

Patient Management

Add, view, and search patient details and medical records

Appointment Booking

Book and cancel appointments with real-time availability checks

Prescription Handling

Manage prescription info and display full clinic summaries

PSP Modules 1–3

Arrays, methods, loops, and conditionals — no advanced Java





Advantages of This Project

The Clinic Appointment and Prescription Manager offers numerous benefits for both learning and practical application.



Real-World Impact

Addresses practical needs in the healthcare domain.



Core Programming Skills

Applies fundamental concepts in a practical setting.



Modular & Reusable

Promotes clean code with reusable methods.



Efficient Data Handling

Manages data effectively using arrays.



Intuitive Interface

User-friendly console with a menu-driven approach.



Maintainable Code

Designed for clarity and ease of maintenance.



Scalable Future

Provides a strong base for future expansion.



Curriculum Alignment

Directly supports BTech 1st-year PSP learning goals.



Enhanced Thinking

Develops crucial problem-solving and computational abilities.



Practical Experience

Hands-on with method overloading and linear search.

The Problem

Manual clinic management makes it difficult to track patient records, appointment availability, and prescription details efficiently.

Paper-based systems lead to errors, delays, and missed appointments — a real challenge for small clinics.

Project Objectives

1 Patient Information

Maintain and search patient records efficiently using arrays

2 Appointment Control

Book and cancel slots with availability validation

3 Prescription Management

Store and display prescription details per patient

4 PSP Concept Integration

Demonstrate Modules 1, 2 & 3 in a real-world context

Module-Wise Concept Mapping

Every feature of the system maps directly to a PSP module — keeping implementation strictly within 1st-year curriculum boundaries.



Fundamentals

Variables, data types, operators, Scanner, console I/O, algorithm design and flowchart



Control Flow

if-else, switch, for / while / do-while loops, break and continue



Modular Programming

Methods with parameters & return values, method overloading, arrays, linear search, basic sorting

System Design & Data Structures

Arrays Used to Store Clinic Data

Array	Sample Values
patientId[]	{101, 102, 103, 104, 105}
patientName[]	{"Rahul", "Anjali", "Arjun", "Priya", "Kiran"}
patientAge[]	{21, 25, 30, 22, 28}
doctorName[]	{"Dr. Sharma", "Dr. Mehta", "Dr. Rao"}
appointmentStatus[]	{"Booked", "Available", "Booked"}



- ❑ Fixed-size arrays with capacity of 5–10 records. No ArrayList or collections used.

Algorithm & Flowchart

Algorithm Steps

01

Initialize Arrays

Set up patient and appointment arrays with default data

02

Display Main Menu

Show options using `switch` statement

03

Read & Validate

Accept user choice via `Scanner`, check valid range

04

Execute Operation

Call appropriate method — add, search, book, cancel

05

Loop or Exit

Return to menu with `while` loop or exit on choice 10



Implementation: Key Methods

addPatient()

Adds new patient using `Scanner` input and stores into parallel arrays at next available index

searchPatient(int id)

Linear search through `patientId[]` — returns index if found, -1 if not.

Overloaded as `searchPatient(String name)` for name-based lookup

bookAppointment()

Validates slot status with `if-else` before updating `appointmentStatus[]` to "Booked"

cancelAppointment()

Resets slot to "Available" after confirmation using conditional checks

displayPatients()

Iterates all records using a `for` loop and prints formatted patient data to console

Java Implementation Overview

Core Program Structure — PSP Concept Map



Scanner Input

Module 1 — all user input via `Scanner` class



Arrays

Module 3 — parallel arrays for patients & appointments



Switch & if-else

Module 2 — menu control and input validation



Loops

Module 2 & 3 — `for` / `while` for iteration and searching



Methods & Overloading

Module 3 — reusable methods with parameters and return values

Strictly Beginner-Friendly

This project is intentionally scoped to PSP Modules 1–3 only. The following advanced concepts are **not used**:

- **×** ArrayList or any Java Collections
- **×** Inheritance or Interfaces
- **×** Exception Handling (try-catch)
- **×** File I/O or database connections
- **×** GUI frameworks or external libraries
- **×** StringBuilder or advanced String APIs

Complete Java Code

```
import java.util.Scanner;

public class ClinicManager {

    // --- Data Structures ---
    // Arrays to store patient information.
    // Capacity is fixed to 10 as per PSP Module 3 constraints.
    static int[] patientId = new int[10];
    static String[] patientName = new String[10];
    static int[] patientAge = new int[10];

    // Arrays to manage doctor appointments.
    static String[] doctorName = {"Dr. Sharma", "Dr. Mehta", "Dr. Rao"}; // Fixed doctors
    // Each element corresponds to a doctor: 0=Dr. Sharma, 1=Dr. Mehta, 2=Dr. Rao
    static String[] appointmentStatus = {"Available", "Available", "Available"}; // Initial status

    // Counter for the number of registered patients
    static int patientCount = 0;

    // Scanner object for user input
    static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        // --- Initial Data (Optional for testing) ---
        // addPatient("Rahul", 25);
        // addPatient("Priya", 30);
        // bookAppointment(patientId[0], 0); // Rahul books Dr. Sharma
        // bookAppointment(patientId[1], 1); // Priya books Dr. Mehta

        int choice;
        // Main menu loop (Module 2: while loop)
        do {
            displayMainMenu();
            System.out.print("Enter your choice: ");
            choice = scanner.nextInt(); // Module 1: Scanner input
            scanner.nextLine(); // Consume newline character

            // Module 2: switch statement for menu navigation
            switch (choice) {
                case 1:
                    addPatient();
                    break;
                case 2:
                    System.out.print("Enter patient ID to search: ");
                    int id = scanner.nextInt();
                    scanner.nextLine();
                    int patientIndexById = searchPatient(id);
                    if (patientIndexById != -1) {
                        System.out.println("Patient Found: ID: " + patientId[patientIndexById] + ", Name: " +
patientName[patientIndexById] + ", Age: " + patientAge[patientIndexById]);
                    } else {
                        System.out.println("Patient not found with ID: " + id);
                    }
                    break;
                case 3:
                    System.out.print("Enter patient name to search: ");
                    String name = scanner.nextLine();
                    int patientIndexByName = searchPatient(name); // Module 3: Method overloading
                    if (patientIndexByName != -1) {
                        System.out.println("Patient Found: ID: " + patientId[patientIndexByName] + ", Name: " +
patientName[patientIndexByName] + ", Age: " + patientAge[patientIndexByName]);
                    } else {
                        System.out.println("Patient not found with name: " + name);
                    }
                    break;
                case 4:
                    bookAppointment();
                    break;
                case 5:
                    cancelAppointment();
                    break;
                case 6:
                    displayPatients();
                    break;
                case 7:
                    System.out.println("Exiting Clinic Appointment and Prescription Manager. Goodbye!");
                    break;
                default:
                    System.out.println("Invalid choice. Please try again.");
            }
            System.out.println("\nPress Enter to continue...");
            scanner.nextLine(); // Wait for user to press Enter
        } while (choice != 7); // Loop continues until user chooses to exit

        scanner.close(); // Close the scanner resource
    }

    // --- Helper Methods ---

    // Displays the main menu options to the user
    public static void displayMainMenu() {
        System.out.println("\n--- Clinic Manager Menu ---");
        System.out.println("1. Add New Patient");
        System.out.println("2. Search Patient by ID");
        System.out.println("3. Search Patient by Name");
        System.out.println("4. Book Appointment");
        System.out.println("5. Cancel Appointment");
        System.out.println("6. Display All Patients");
        System.out.println("7. Exit");
        System.out.println("-----");
    }

    // Module 3: Method to add a new patient
    public static void addPatient() {
        if (patientCount < patientId.length) { // Module 2: Conditional check for array capacity
            System.out.println("\n--- Add New Patient ---");
            System.out.print("Enter patient name: ");
            String newName = scanner.nextLine();
            System.out.print("Enter patient age: ");
            int newAge = scanner.nextInt();
            scanner.nextLine(); // Consume newline

            // Generate a simple unique patient ID
            patientId[patientCount] = 1001 + patientCount;
            patientName[patientCount] = newName;
            patientAge[patientCount] = newAge;

            System.out.println("Patient added successfully! ID: " + patientId[patientCount]);
            patientCount++; // Increment patient count
        } else {
            System.out.println("Clinic is full. Cannot add more patients.");
        }
    }

    // Module 3: Method to search patient by ID (linear search)
    public static int searchPatient(int id) {
        for (int i = 0; i < patientCount; i++) { // Module 2: for loop for iteration
            if (patientId[i] == id) { // Module 2: Conditional check
                return i; // Return index if found
            }
        }
        return -1; // Not found
    }

    // Module 3: Overloaded method to search patient by name (linear search)
    public static int searchPatient(String name) {
        for (int i = 0; i < patientCount; i++) {
            // Using equalsIgnoreCase for a more user-friendly search
            if (patientName[i].equalsIgnoreCase(name)) {
                return i; // Return index if found
            }
        }
        return -1; // Not found
    }

    // Module 3: Method to book an appointment
    public static void bookAppointment() {
        System.out.println("\n--- Book Appointment ---");
        System.out.print("Enter patient ID: ");
        int pId = scanner.nextInt();
        scanner.nextLine();

        int patientIdx = searchPatient(pId);
        if (patientIdx == -1) {
            System.out.println("Patient not found.");
            return;
        }

        System.out.println("Available Doctors:");
        for (int i = 0; i < doctorName.length; i++) {
            System.out.println((i + 1) + ". " + doctorName[i] + " (" + appointmentStatus[i] + ")");
        }
        System.out.print("Select doctor (1-" + doctorName.length + ") ");
        int doctorChoice = scanner.nextInt();
        scanner.nextLine();

        if (doctorChoice >= 1 && doctorChoice <= doctorName.length) {
            int docIdx = doctorChoice - 1;
            if (appointmentStatus[docIdx].equals("Available")) { // Module 2: Conditional check
                appointmentStatus[docIdx] = "Booked by Patient ID: " + pId + " (" + patientName[patientIdx] + ")";
                System.out.println("Appointment booked successfully for " + patientName[patientIdx] + " with " +
doctorName[docIdx] + ".");
            } else {
                System.out.println("Sorry, " + doctorName[docIdx] + " is already " + appointmentStatus[docIdx] + ". Please choose
another doctor.");
            }
        } else {
            System.out.println("Invalid doctor selection.");
        }
    }

    // Module 3: Method to cancel an appointment
    public static void cancelAppointment() {
        System.out.println("\n--- Cancel Appointment ---");
        System.out.print("Enter patient ID whose appointment you want to cancel: ");
        int pId = scanner.nextInt();
        scanner.nextLine();

        int patientIdx = searchPatient(pId);
        if (patientIdx == -1) {
            System.out.println("Patient not found.");
            return;
        }

        boolean foundAppointment = false;
        for (int i = 0; i < doctorName.length; i++) { // Module 2: for loop
            if (appointmentStatus[i].contains("Booked by Patient ID: " + pId)) { // Module 2: Conditional check
                System.out.println("Found Appointment for " + patientName[patientIdx] + " with " + doctorName[i] + ". Do you
want to cancel? (yes/no)");
                String confirm = scanner.nextLine();
                if (confirm.equalsIgnoreCase("yes")) { // Module 2: Conditional check
                    appointmentStatus[i] = "Available";
                    System.out.println("Appointment with " + doctorName[i] + " cancelled successfully.");
                } else {
                    System.out.println("Cancellation aborted.");
                }
                foundAppointment = true;
                break; // Exit loop after finding and processing the appointment
            }
        }
        if (!foundAppointment) {
            System.out.println("No appointment found for Patient ID: " + pId);
        }
    }

    // Module 3: Method to display all registered patients
    public static void displayPatients() {
        System.out.println("\n--- All Registered Patients ---");
        if (patientCount == 0) {
            System.out.println("No patients registered yet.");
            return;
        }
        System.out.printf("%-5s %-15s %-5s\n", "ID", "Name", "Age"); // Formatted output (Module 1)
        System.out.println("-----");
        for (int i = 0; i < patientCount; i++) { // Module 2: for loop for iteration
            System.out.printf("%-5d %-15s %-5d\n", patientId[i], patientName[i], patientAge[i]);
        }

        System.out.println("\n--- Current Doctor Appointments ---");
        System.out.printf("%-15s %-30s\n", "Doctor", "Status");
        System.out.println("-----");
        for (int i = 0; i < doctorName.length; i++) {
            System.out.printf("%-15s %-30s\n", doctorName[i], appointmentStatus[i]);
        }
    }
}
```

Sample Output & Execution

Main Menu Console Output

===== CLINIC MANAGER =====

1. Add Patient
2. Display Patients
3. Search Patient
4. Book Appointment
5. Cancel Appointment
6. Add Prescription
7. Display Prescription
8. Available Slots
9. Clinic Summary
10. Exit

Enter choice: 1

Add Patient — Sample Run

Enter Patient ID: 101

Enter Name: Rahul

Enter Age: 21

✓ Patient added successfully!

Enter Patient ID: 102

Enter Name: Anjali

Enter Age: 25

✓ Patient added successfully!

Search Patient — Overloaded Method

Search by ID: searchPatient(101)

→ Found: Rahul, Age 21

Search by Name: searchPatient("Anjali")

→ Found: ID 102, Age 25



Conclusion & Future Work

CONCLUSION

- Successfully integrates PSP Modules 1, 2, and 3, demonstrating foundational programming principles.
- Provides a practical, modular solution for basic clinic management, including patient, doctor, and appointment handling.
- Emphasizes learning outcomes in structured programming, data management, and problem decomposition.
- Highlights the importance of modular design for maintainability and scalability in software development.

FUTURE ENHANCEMENTS

- Implement a Graphical User Interface (GUI) using Java Swing or JavaFX for enhanced user interaction.
- Integrate with a database (e.g., MySQL, PostgreSQL) for persistent data storage.
- Add file-based data persistence (e.g., object serialization) as an alternative storage option.
- Develop advanced reporting and analytics features for clinic performance insights.
- Incorporate multi-user support with authentication and role-based access control.
- Automate prescription refill processes and manage inventory.
- Introduce appointment reminders and notifications via SMS or email.
- Integrate with secure payment gateways for billing and invoicing.

Thank You

We sincerely appreciate your time and attention today. Your engagement and insights are invaluable.

Presenter: Sundi Manilatha

Course: Programming for Problem Solving (PSP) – Java

Project: Clinic Appointment and Prescription Manager